

Testing Plan

Rencana Uji Integrasi Protokol AAA

Bacarita — Authentication • Authorization • Accounting

Daftar Isi

1. Gambaran Umum Pengujian	2
1.1. Ruang Lingkup	2
1.2. Metodologi	2
1.3. Konvensi Status Hasil	2
2. Authentication Tests	2
2.1. TC-AUTH-01: Login Sukses dan Token HttpOnly Cookie	2
2.2. TC-AUTH-02: Login dengan Kredensial Salah	3
2.3. TC-AUTH-03: Account Lockout setelah 5 Kali Gagal	3
2.4. TC-AUTH-04: Rate Limiting Login	4
2.5. TC-AUTH-05: Verifikasi Token Hash di Database	4
2.6. TC-AUTH-06: Logout Invalidasi Token	5
2.7. TC-AUTH-07: Validasi Kompleksitas Password	5
3. Authorization Tests	6
3.1. TC-AUTHZ-01: Akses Endpoint dengan Peran Benar	6
3.2. TC-AUTHZ-02: Akses Ditolak untuk Peran yang Salah	6
3.3. TC-AUTHZ-03: Akses Tanpa Token	7
3.4. TC-AUTHZ-04: Role Elevation Attack	7
4. Accounting Tests	7
4.1. TC-ACCT-01: Audit Log Login Sukses	7
4.2. TC-ACCT-02: Audit Log Login Gagal	8
4.3. TC-ACCT-03: Audit Log Account Lockout	8
4.4. TC-ACCT-04: Audit Log Logout	8
5. Security Regression Tests	9
6. Matriks Pengujian	10
6.1. Kriteria Kelulusan	10
7. Referensi Kode Uji	10

1. Gambaran Umum Pengujian

1.1. Ruang Lingkup

Dokumen ini mendefinisikan rencana uji integrasi untuk memvalidasi implementasi protokol keamanan AAA (Authentication, Authorization, Accounting) pada sistem Bacarita setelah remediasi berikut diterapkan:

Remediasi	Deskripsi	Kategori AAA
AUTH-05	HttpOnly + Secure + SameSite cookie via Next.js server-side	Authentication
AUTH-01/02	Rate limiting 5 req/60 dtk per IP pada semua endpoint login	Authentication
AUTH-03	Token tersimpan sebagai SHA-256 hash di database	Authentication
USER-03	Account lockout 15 menit setelah 5 kali login gagal	Authentication
VA-08	Validasi password minimal 8 karakter di semua DTO	Authentication
AUTH-06	Audit log event login, logout, dan lockout	Accounting

Tabel 1: Remediasi yang diuji

1.2. Metodologi

- **Jenis Uji:** Integration Testing (black-box & white-box)
- **Target:** NestJS REST API (backend) + Next.js session route (frontend)
- **Alat:** Supertest (unit/integration NestJS), HTTPie/curl (manual), Jest (test runner)
- **Lingkungan:** `NODE_ENV=test` dengan database MySQL terpisah

1.3. Konvensi Status Hasil

Status	Simbol	Makna
PASS	✓	Sistem berperilaku sesuai ekspektasi keamanan
FAIL	✗	Sistem tidak memenuhi ekspektasi; perlu perbaikan
SKIP	—	Belum dapat diuji (dependensi belum tersedia)

Tabel 2: Konvensi status hasil uji

2. Authentication Tests

2.1. TC-AUTH-01: Login Sukses dan Token HttpOnly Cookie

Tujuan: Memastikan token JWT dikirim via HttpOnly cookie, bukan di JSON body yang dapat diakses JavaScript.

Prasyarat: Akun guru terdaftar di database test dengan password valid.

Langkah Uji:

```
POST /auth/teachers/login
Content-Type: application/json

{ "email": "guru@test.com", "password": "Password1!" }
```

Ekspektasi:

No	Verifikasi	Ekspektasi
1	HTTP status code	200 OK
2	Response header Set-Cookie	Ada; mengandung HttpOnly
3	Response header Set-Cookie	Mengandung Secure
4	Response header Set-Cookie	Mengandung SameSite=Strict
5	Response header Set-Cookie	Mengandung Max-Age=3600
6	Response body data.token	Tidak ada token di JSON body
7	Database teachers.token	SHA-256 hash (64 karakter hex), bukan JWT

Tabel 3: TC-AUTH-01: Ekspektasi

2.2. TC-AUTH-02: Login dengan Kredensial Salah

Tujuan: Memastikan sistem menolak kredensial salah dan menginkremen counter gagal.

Langkah Uji:

```
POST /auth/teachers/login
{ "email": "guru@test.com", "password": "passwordsalah" }
```

Ekspektasi:

No	Verifikasi	Ekspektasi
1	HTTP status code	401 Unauthorized
2	Response body message	Pesan generik; tidak membedakan email vs password
3	Database failedLoginAttempts	Bertambah 1 dari nilai sebelumnya
4	Database lockedUntil	NULL (belum terkunci, baru 1x gagal)

Tabel 4: TC-AUTH-02: Ekspektasi

2.3. TC-AUTH-03: Account Lockout setelah 5 Kali Gagal

Tujuan: Memastikan akun terkunci otomatis setelah 5 kali login gagal berturut-turut.

Langkah Uji: Kirim 5 kali request dengan password salah, kemudian kirim request ke-6 dengan password benar.

```
# 5x berturut-turut:
POST /auth/teachers/login
{ "email": "guru@test.com", "password": "salah" }

# Request ke-6 dengan password benar:
POST /auth/teachers/login
{ "email": "guru@test.com", "password": "Password1!" }
```

Ekspektasi:

No	Verifikasi	Ekspektasi
1	HTTP status kode setelah percobaan ke-5	401 Unauthorized
2	Database failedLoginAttempts setelah ke-5	5
3	Database lockedUntil setelah ke-5	Terisi: waktu sekarang + 15 menit
4	HTTP status kode request ke-6 (benar)	423 Locked (atau 401)
5	Response body request ke-6	Menunjukkan akun terkunci
6	Setelah 15 menit, request dengan benar	200 OK; failedLoginAttempts reset ke 0

Tabel 5: TC-AUTH-03: Ekspektasi

2.4. TC-AUTH-04: Rate Limiting Login

Tujuan: Memastikan endpoint login menolak request lebih dari 5 kali per 60 detik dari IP yang sama.

Langkah Uji: Kirim 6 request login dalam waktu < 60 detik dari IP yang sama.

```
# Loop 6x dalam < 60 detik
for i in $(seq 1 6); do
    curl -s -o /dev/null -w "%{http_code}\n" \
        -X POST http://localhost:3000/auth/teachers/login \
        -H "Content-Type: application/json" \
        -d '{"email": "guru@test.com", "password": "test"}'
done
```

Ekspektasi:

No	Verifikasi	Ekspektasi
1	HTTP status request ke-1 s/d ke-5	200 atau 401 (tergantung kredensial)
2	HTTP status request ke-6	429 Too Many Requests
3	Response header Retry-After	Ada; menunjukkan sisa waktu tunggu
4	Setelah 60 detik, request baru	Dapat diproses normal kembali

Tabel 6: TC-AUTH-04: Ekspektasi

2.5. TC-AUTH-05: Verifikasi Token Hash di Database

Tujuan: Memastikan yang tersimpan di database adalah SHA-256 hash, bukan JWT asli.

Langkah Uji: Login sukses, kemudian periksa nilai kolom token di database.

Ekspektasi:

No	Verifikasi	Ekspektasi
1	Panjang nilai token di DB	64 karakter (SHA-256 hex)
2	Format nilai token	Hexadecimal <code>[0-9a-f]{64}</code> ; bukan JWT format <code>xxx.yyy.zzz</code>
3	SHA-256(JWT dari cookie) == token di DB	Sama persis
4	JWT asli dari cookie ada di DB	Tidak ada

Tabel 7: TC-AUTH-05: Ekspektasi

2.6. TC-AUTH-06: Logout Invalidasi Token

Tujuan: Memastikan logout menghapus hash token dari database dan token lama tidak dapat digunakan.

Langkah Uji:

```
# 1. Login
POST /auth/teachers/login → simpan token dari cookie

# 2. Gunakan token untuk akses resource
GET /auth/me
Authorization: Bearer <token> → 200 OK

# 3. Logout
POST /auth/teachers/logout
Authorization: Bearer <token> → 200 OK

# 4. Coba gunakan token lama setelah logout
GET /auth/me
Authorization: Bearer <token> → harus ditolak
```

Ekspektasi:

No	Verifikasi	Ekspektasi
1	Database token setelah logout	NULL
2	GET /auth/me dengan token lama setelah logout	401 Unauthorized
3	Cookie terhapus setelah logout	Set-Cookie: token=; Max-Age=0

Tabel 8: TC-AUTH-06: Ekspektasi

2.7. TC-AUTH-07: Validasi Kompleksitas Password

Tujuan: Memastikan DTO menolak password kurang dari 8 karakter saat login atau registrasi.

Langkah Uji:

```
POST /auth/students/login
{ "username": "siswa01", "password": "abc" }
```

Ekspektasi:

No	Verifikasi	Ekspektasi
1	HTTP status code	400 Bad Request
2	Response body errors	Mengandung pesan validasi minLength
3	Password 8 karakter atau lebih	Lolos validasi (diproses normal)

Tabel 9: TC-AUTH-07: Ekspektasi

3. Authorization Tests

3.1. TC-AUTHZ-01: Akses Endpoint dengan Peran Benar

Tujuan: Memastikan setiap peran hanya dapat mengakses endpoint yang sesuai.

Langkah Uji: Login dengan setiap peran, akses endpoint yang seharusnya diizinkan.

Peran	Endpoint	Ekspektasi	Hasil
Teacher	GET /levels (CRUD level)	200 OK	—
Student	POST /test-sessions (mulai sesi)	200 OK	—
Parent	GET /dashboard/parent	200 OK	—
Admin	POST /levels (buat level)	200 OK	—
Curator	PATCH /stories/:id/approve	200 OK	—

Tabel 10: TC-AUTHZ-01: Akses endpoint sesuai peran

3.2. TC-AUTHZ-02: Akses Ditolak untuk Peran yang Salah

Tujuan: Memastikan sistem menolak akses ketika peran tidak sesuai dekorator @Auth().

Langkah Uji: Login sebagai Student, coba akses endpoint yang hanya boleh untuk Teacher.

```
# Login sebagai student
POST /auth/students/login → token siswa

# Coba akses endpoint teacher-only
POST /levels
Authorization: Bearer <student-token>
```

Ekspektasi:

No	Verifikasi	Ekspektasi
1	HTTP status code	403 Forbidden
2	Student tidak dapat POST /levels	403 Forbidden
3	Student tidak dapat PATCH /stories/:id/approve	403 Forbidden
4	Student tidak dapat GET /dashboard/admin	403 Forbidden
5	Parent tidak dapat melihat siswa di luar anaknya	404 atau 403

Tabel 11: TC-AUTHZ-02: Penolakan akses lintas peran

3.3. TC-AUTHZ-03: Akses Tanpa Token

Tujuan: Memastikan semua endpoint terproteksi menolak request tanpa token.

Langkah Uji: Kirim request ke endpoint terproteksi tanpa header Authorization.

```
GET /auth/me
# Tanpa Authorization header
```

Ekspektasi:

No	Verifikasi	Ekspektasi
1	HTTP status code	401 Unauthorized
2	Token kedaluwarsa	401 Unauthorized
3	Token dengan signature tidak valid (dimodifikasi)	401 Unauthorized
4	Token valid tapi sudah logout (hash tidak ada di DB)	401 Unauthorized

Tabel 12: TC-AUTHZ-03: Penolakan request tanpa/token tidak valid

3.4. TC-AUTHZ-04: Role Elevation Attack

Tujuan: Memastikan manipulasi payload JWT tidak dapat menaikkan hak akses.

Langkah Uji: Dekode JWT student, ubah role menjadi admin, encode ulang tanpa secret yang benar.

```
# Token asli siswa (decoded payload):
{ "id": "abc", "role": "student", "iat": ..., "exp": ... }

# Coba buat token palsu dengan role "admin":
# (ditandatangani dengan secret acak, bukan JWT_SECRET yang benar)
```

Ekspektasi:

No	Verifikasi	Ekspektasi
1	Token palsu dengan role admin dikirim ke endpoint admin-only	401 Unauthorized
2	Alasan penolakan	Signature verification gagal (jwtService.verify throw)
3	Token valid siswa dengan hash tidak ada di DB	401 Unauthorized

Tabel 13: TC-AUTHZ-04: Role elevation attack

4. Accounting Tests

4.1. TC-ACCT-01: Audit Log Login Sukses

Tujuan: Memastikan setiap login sukses dicatat di tabel auth_audit_logs.

Langkah Uji: Login sukses sebagai guru, kemudian periksa tabel audit log.

Ekspektasi:

No	Verifikasi	Ekspektasi
1	Record baru di auth_audit_logs	Ada
2	Kolom event	LOGIN_OK
3	Kolom userId	ID guru yang bersangkutan
4	Kolom role	teacher
5	Kolom ipAddress	IP address klien (bukan null)
6	Kolom createdAt	Timestamp login (akurat ± 5 detik)

Tabel 14: TC-ACCT-01: Audit log login sukses

4.2. TC-ACCT-02: Audit Log Login Gagal

Tujuan: Memastikan percobaan login gagal dicatat untuk keperluan monitoring anomali.

Ekspektasi:

No	Verifikasi	Ekspektasi
1	Record di auth_audit_logs setelah gagal	Ada
2	Kolom event	LOGIN_FAIL
3	Kolom userId	ID pengguna yang dicoba (atau null jika user tidak ada)
4	Log tidak mengandung password yang dimasukkan	Benar; tidak ada data password di log

Tabel 15: TC-ACCT-02: Audit log login gagal

4.3. TC-ACCT-03: Audit Log Account Lockout

Tujuan: Memastikan event lockout dicatat saat akun dikunci otomatis.

Ekspektasi:

No	Verifikasi	Ekspektasi
1	Record LOCKED di audit log setelah percobaan ke-5	Ada
2	Kolom event	LOCKED
3	Urutan log untuk akun yang sama	LOGIN_FAIL $\times 5 \rightarrow$ LOCKED

Tabel 16: TC-ACCT-03: Audit log lockout

4.4. TC-ACCT-04: Audit Log Logout

Tujuan: Memastikan logout dicatat dan token hash dihapus dari database.

Ekspektasi:

No	Verifikasi	Ekspektasi
1	Record di auth_audit_logs setelah logout	Ada
2	Kolom event	LOGOUT
3	Kolom token di tabel pengguna setelah logout	NULL

Tabel 17: TC-ACCT-04: Audit log logout

5. Security Regression Tests

Uji regresi ini memastikan remediasi tidak merusak fungsionalitas yang ada.

ID	Skenario Regresi	Ekspektasi
REG-01	Login semua 5 peran (teacher, student, parent, admin, curator) tetap berfungsi	200 OK semua
REG-02	GET /auth/me setelah login masih mengembalikan profil yang benar	200 OK + data profil
REG-03	Sesi tes siswa (POST /test-sessions) masih dapat dibuat	200 OK
REG-04	Dashboard guru masih dapat diakses dengan token guru yang valid	200 OK
REG-05	Kurasi cerita oleh kurator (PATCH approve/reject) masih berfungsi	200 OK
REG-06	Upload gambar cerita oleh guru masih berfungsi	200 OK + file tersimpan
REG-07	Paginasi pada listing cerita masih berfungsi	200 OK + pagination metadata
REG-08	End-to-end: siswa login → pilih cerita → mulai sesi → kirim hasil	Semua 200 OK

Tabel 18: Security regression test cases

6. Matriks Pengujian

ID	Nama	Auth	Authz	Acct	Prioritas
TC-AUTH-01	Login sukses + HttpOnly cookie	✓	–	–	P1
TC-AUTH-02	Login gagal + counter	✓	–	–	P1
TC-AUTH-03	Account lockout 5x gagal	✓	–	✓	P1
TC-AUTH-04	Rate limiting 5 req/60s	✓	–	–	P1
TC-AUTH-05	Token hash SHA-256 di DB	✓	–	–	P2
TC-AUTH-06	Logout invalidasi token	✓	–	✓	P1
TC-AUTH-07	Validasi password MinLength	✓	–	–	P3
TC-AUTHZ-01	Akses endpoint peran benar	–	✓	–	P1
TC-AUTHZ-02	Akses ditolak peran salah	–	✓	–	P1
TC-AUTHZ-03	Akses tanpa token	✓	✓	–	P1
TC-AUTHZ-04	Role elevation attack	–	✓	–	P2
TC-ACCT-01	Audit log login sukses	–	–	✓	P2
TC-ACCT-02	Audit log login gagal	–	–	✓	P2
TC-ACCT-03	Audit log lockout	–	–	✓	P2
TC-ACCT-04	Audit log logout	–	–	✓	P2
REG-01-08	Regression tests fungsionalitas	✓	✓	–	P1

Tabel 19: Matriks pengujian lengkap

6.1. Kriteria Kelulusan

Implementasi dinyatakan **lulus** apabila:

- Seluruh test case Prioritas P1 bernilai PASS
- Minimal 80% test case Prioritas P2 bernilai PASS
- Tidak ada regression test yang bernilai FAIL
- Tidak ada kerentanan Kritis baru yang ditemukan selama pengujian

7. Referensi Kode Uji

Contoh struktur test Supertest untuk NestJS:

```
// test/auth.integration.spec.ts
describe('Auth Integration - Rate Limiting', () => {
  it('TC-AUTH-04: harus mengembalikan 429 pada request ke-6 dalam 60 detik', async
    () => {
      for (let i = 0; i < 5; i++) {
        await request(app.getHttpServer())
          .post('/auth/teachers/login')
          .send({ email: 'guru@test.com', password: 'salah' });
      }
      const res = await request(app.getHttpServer())
```

```

        .post('/auth/teachers/login')
        .send({ email: 'guru@test.com', password: 'salah' });

        expect(res.status).toBe(429);
    });
});

describe('Auth Integration - Account Lockout', () => {
    it('TC-AUTH-03: akun terkunci setelah 5x gagal, menolak login benar ke-6', async () => {
        for (let i = 0; i < 5; i++) {
            await request(app.getHttpServer())
                .post('/auth/teachers/login')
                .send({ email: 'guru@test.com', password: 'salah' });
        }
        const res = await request(app.getHttpServer())
            .post('/auth/teachers/login')
            .send({ email: 'guru@test.com', password: 'Password1!' });

        expect([401, 423]).toContain(res.status);
    });
});

describe('Auth Integration - Token Hash', () => {
    it('TC-AUTH-05: token di DB adalah SHA-256 hash, bukan JWT asli', async () => {
        const loginRes = await request(app.getHttpServer())
            .post('/auth/teachers/login')
            .send({ email: 'guru@test.com', password: 'Password1!' });

        const jwt = loginRes.headers['set-cookie'][0].match(/token=([^;]+)/)[1];
        const hash = createHash('sha256').update(jwt).digest('hex');
        const teacher = await teacherRepo.findOne({ where: { email: 'guru@test.com' } });

        expect(teacher.token).toBe(hash);
        expect(teacher.token).not.toBe(jwt);
        expect(teacher.token).toMatch(/^[0-9a-f]{64}$/);
    });
});

```